Unit 3: Design Patterns I - Creational Patterns

Task ‘Implementing the Factory Method Pattern’

The Factory Method Pattern supports the separation of object creation from the client code hence improving flexibility and maintainability, in this car manufacturing system, the client creates cars via factories instead of directly instantiating classes, this follows the Open/Closed Principle because new car types can be added without modifying existing code.

(Lott and Phillips, 2021).

The Car abstract class defines the common operation drive(). Each concrete class (Sedan, SUV, and Hatchback) provides its own implementation. The CarFactory abstract class declares the factory method create_car(), while each concrete factory creates a specific car object.

The client interacts through abstract interfaces after the factory creation; accordingly, the client code does not rely on concrete classes, resulting in an easier system, extendibility and maintainability. If the company later wants to introduce an ElectricSUV, a new factory and class can be added without changing the existing client logic, this demonstrates how the Factory Method Pattern supports scalable software design in manufacturing systems.

Client Code of the Factory Method Demonstration:

```python
factories = [SedanFactory(), SUVFactory(), HatchbackFactory()]
```

```
for factory in factories:
    car = factory.create_car()
    car.drive()
```

(Lott and Phillips, 2021).

Explanation

The client code primarily interacts with the abstract types CarFactory and Car, alllthgoh concrete factories are instantiated initially.

The client code does not directly instantiate objects such as Sedan() or SUV().

Each factory decides which concrete car object to produce through create_car().

The drive() method is called using the common Car interface.

This demonstrates the Factory Method Pattern because object creation is delegated to factory subclasses. The client remains independent of the concrete car classes, improving flexibility and scalability.

(Lott and Phillips, 2021).

Result

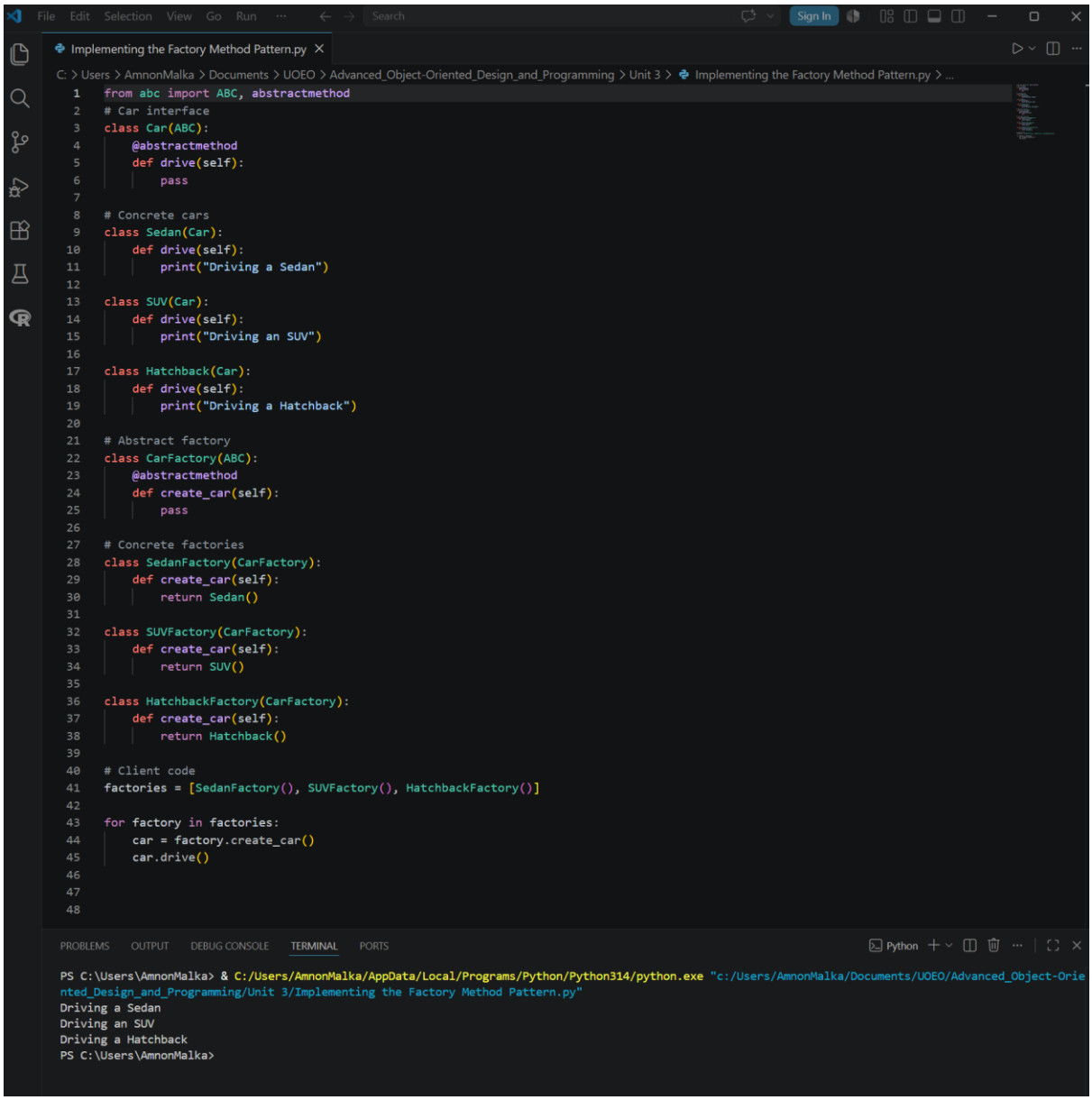

Figure 1: Test results

References

- Lott, S. F. & Phillips, D. (2021) *Python Object-Oriented Programming: Build robust and maintainable object-oriented Python applications and libraries*. Fourth edition. [Online]. Birmingham: Packt Publishing. Available at: https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_proquest_ebookcentral_EBC6680969 [Accessed: 14 May 2026]

Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.

Appendix A: Full Python Implementation

Implementing_the_Factory_Method_Pattern.py